REG: 222009343

YEAR III Computer and Software Engineering

Written part

(1) Dependencies I had used and why are:

* I used http package (http: ^1.2.0): so I chose this package because it provides simple and ready-made methods for making HTTP requests such as http.get(), http.post(), http.put() and http.delete() - these methods match exactly the CRUD operations needed to communicate with the JSONPlaceholder REST API. It is the official and most commonly used package for API consumption in flutter.

(2) How I handle API exceptions: API exceptions are handled using try/catch blocks around every network call. There are three levels of error handling:

* Network errors: Caught using http.ClientException which fires when there is no Internet connection or the server is unreachable. The user sees a message like "please check your Internet connection"
* Server errors: After successful connection we check response.statusCode. If it is not 200 or 201 we throw a custom Exception with the status code.
* Unexpected errors:
* Missing / invalid fields: If the JSON is missing expected fields like id, title or body the Post.fromJson() factory constructor will throw an error which is also caught by the catch(e) block.

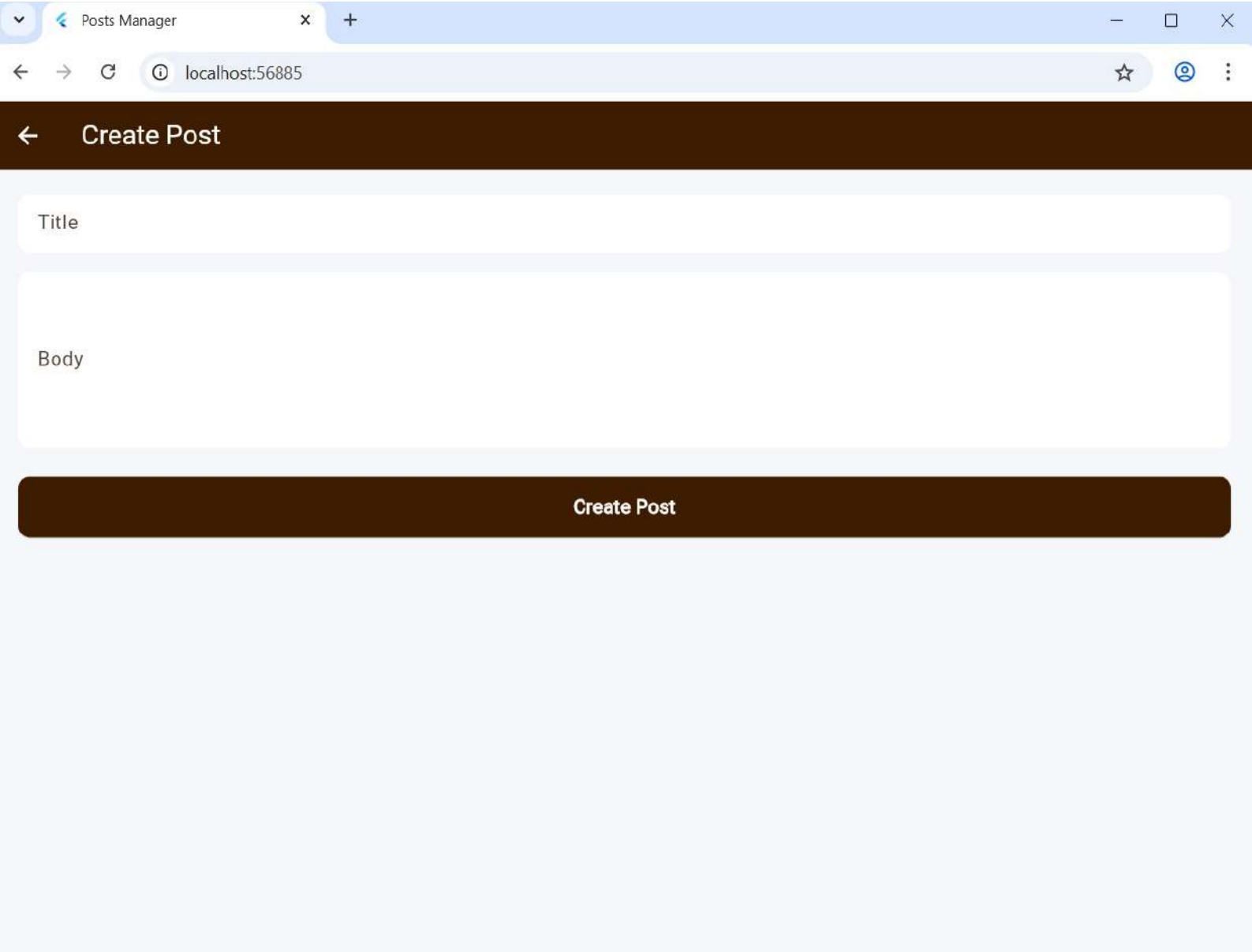
(3) FUTURE Builder and UI depends on it:

FutureBuilder is a flutter widget that listens to a future and automatically rebuilds the UI whenever the future's state changes. It takes two parameters: future, which is the async API call and builder which returns different widgets based on the asyncSnapshot state.

In my app we used FutureBuilder<List<Post>> in the PostListScreen. It handles three states:

- Connection state. waiting → displays a CircularProgressIndicator (loading spinner) while the API call is in progress
- Snapshot.hasError → displays a red error message with a Retry button if the API called failed
- Snapshot.hasData → displays the ListView.builder with all posts when data is successfully returned.

The entire body of PostsListScreen depends on the FutureBuilder, nothing is shown to the user until the future resolves. The future is stored in _postsFuture and is refreshed by calling setState() inside the _refresh() method


Posts Manager
localhost:56885
Create Post
Title
Body
Create Post

Posts Manager

localhost:56885

Posts Manager

1
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit
suscipit recusandae consequuntur expedita et cum

2
qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque

3
ea molestias quasi exercitationem repellat qui ipsa sit aut
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad

4
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa

5
nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est

+ New Post

dolorem eum magni eos aperiam quia

Post deleted successfully

Posts Manager

localhost:56885

Post Details

Post #95 • User 10

id minus libero illum nam ad officiis

---

earum voluptatem facere provident blanditiis velit laboriosam
pariatur accusamus odio saepe
cumque dolor qui a dicta ab doloribus consequatur omnis
corporis cupiditate eaque assumenda ad nesciunt

Posts Manager

localhost:56885

Edit Post

Title

id minus libero illum nam ad officiis

Body

earum voluptatem facere provident blanditiis velit laboriosam
pariatur accusamus odio saepe
cumque dolor qui a dicta ab doloribus consequatur omnis
corporis cupiditate eaque assumenda ad nesciunt

Save Changes

Posts Manager

localhost:56885

Posts Manager

95
id minus libero illum nam ad officiis
earum voluptatem facere provident blanditiis velit laboriosam
pariatur accusamus odio saepe

96
quaerat velit veniam amet cupiditate aut numquam ut sequi
in non odio excepturi sint eum
labore voluptates vitae quia qui et

97
quas fugiat ut perspiciatis vero provident
eum non blanditiis soluta porro quibusdam voluptas
vel voluptatem qui placeat dolores qui velit aut

98
laboriosam dolor voluptates
doloremque ex facilis sit sint culpa
soluta assumenda eligendi non ut eius

99
temporibus sit alias delectus eligendi possimus magni
quo deleniti praesentium dicta non quod
aut est molestias

100
at nam consequatur ea labore ea harum
cupiditate quo est a modi nesciunt soluta
ipsa voluptas error itaque dicta in

+ New Post